

Documentación del Módulo de Persistencia

¿Cómo funciona?

El módulo de persistencia se encarga exclusivamente de leer y escribir los datos de clientes y productos en archivos JSON. Está separado del resto de la aplicación: los services (`clientesService.ts`, `productoService.ts`) nunca tocan el sistema de archivos directamente, siempre pasan por las funciones de `reader.ts` y `writer.ts`.

Flujo general:

1. Un service necesita datos → llama a `leerClientes()` o `leerProductos()`.
2. El service modifica el arreglo en memoria (agrega, actualiza o elimina un registro).
3. El service llama a `escribirClientes()` o `escribirProductos()` para guardar el arreglo completo de vuelta al archivo JSON.

Toda la lectura y escritura es asíncrona, usando `fs/promises` (`readFile` y `writeFile`).

Archivos utilizados

Archivo	Contenido
<code>src/data/clientes.json</code>	Arreglo de objetos Cliente
<code>src/data/producto.json</code>	Arreglo de objetos Producto

Cada archivo almacena un arreglo JSON plano de los registros. No hay anidamiento ni metadata extra; el archivo completo se sobrescribe en cada escritura.

Lectura de datos

Las funciones `leerClientes()` y `leerProductos()` (en `src/utls/reader.ts`) hacen lo siguiente:

1. Intentan leer el archivo correspondiente con `readFile(..., "utf8")`.
2. Convierten el contenido con `JSON.parse()` y regresan el arreglo.
3. Si algo falla, entran al `catch` y manejan el error según el caso (ver sección de errores abajo).

Escritura de datos

Las funciones `escribirClientes()` y `escribirProductos()` (en `src/utls/writer.ts`) reciben el arreglo completo ya modificado y lo escriben con `writeFile()`, usando `JSON.stringify(data, null, 2)` para que el archivo quede legible (con indentación).

No se hacen escrituras parciales: cada vez que se crea, actualiza o elimina un registro, se vuelve a escribir el arreglo completo en el archivo.

Tipos de errores que se manejan

Dentro de leerClientes() / leerProductos():

- **Archivo inexistente (ENOENT):** si el archivo no existe todavía (por ejemplo, primera vez que corre la app), se crea automáticamente con un arreglo vacío [] y se le avisa al usuario que se generó uno nuevo.
- **JSON corrupto (SyntaxError):** si el archivo existe pero su contenido no es un JSON válido (por ejemplo, se editó a mano y quedó mal formado), se captura el error, se informa por consola y se regresa un arreglo vacío en vez de tronar la aplicación.
- **Otros errores** (por ejemplo, problemas de permisos): se loguean con console.error(error.message) y también se regresa un arreglo vacío, para que el programa pueda seguir corriendo sin caerse.

Dentro de escribirClientes() / escribirProductos():

- Cualquier error durante la escritura (permisos, disco lleno, etc.) se captura con try/catch y se muestra un mensaje claro en consola ("Error al escribir el archivo de clientes/productos:"), sin detener la ejecución del programa.

Además de los errores de persistencia, el módulo de validaciones (src/utls/validaciones.ts) revisa los datos **antes** de que lleguen a escribirse: nombre/apellido vacíos, teléfono y DPI numéricos válidos, dominio de correo permitido (Gmail, Yahoo, Outlook) para clientes; y nombre, código, precio y cantidad válidos para productos. Si algo no cumple, se lanza un Error que el menú captura con su propio try/catch y muestra el mensaje al usuario sin cerrar la aplicación.